

Esercitazione in PySpark

Programmazione

PySpark è l'interfaccia Python di Apache Spark, un framework open source progettato per l'elaborazione distribuita di grandi quantità di dati.

A differenza delle tradizionali librerie Python che elaborano principalmente i dati su una singola macchina, PySpark consente di distribuire le operazioni su più nodi di un cluster, sfruttando le risorse computazionali disponibili.

Questo approccio permette di gestire dataset di grandi dimensioni attraverso operazioni parallele, migliorando la scalabilità dei processi di elaborazione.

PySpark viene utilizzato principalmente nell'ambito dei Big Data, della Data Engineering e del Machine Learning per eseguire attività come pulizia dei dati, trasformazioni, analisi statistiche e costruzione di pipeline.



Una delle caratteristiche fondamentali di PySpark è l'elaborazione distribuita dei dati attraverso il modello Driver-Executor.

Il Driver coordina l'esecuzione del programma, mentre gli Executor eseguono le operazioni sui dati suddivisi in partizioni.

PySpark utilizza il concetto di lazy evaluation, secondo cui molte trasformazioni, come filter(), select() e groupBy(), non vengono eseguite immediatamente, ma vengono organizzate in un piano di esecuzione che viene attivato quando viene richiesta un'azione, come show(), count() o collect().

Questo meccanismo consente al motore Spark di ottimizzare le operazioni e limitare, quando possibile, l'utilizzo delle risorse computazionali.



PySpark mette a disposizione diverse API per la manipolazione e l'analisi dei dati, tra cui DataFrame, Spark SQL e MLlib.

I DataFrame rappresentano strutture tabellari distribuite, organizzate in righe e colonne, sulle quali è possibile applicare filtri, aggregazioni, ordinamenti e collegamenti tra dataset tramite operazioni di join.

Spark SQL permette di interrogare i dati utilizzando istruzioni SQL, mentre MLlib fornisce strumenti per sviluppare modelli di Machine Learning e pipeline di elaborazione.

PySpark supporta inoltre diversi formati di dati, tra cui CSV, JSON e Parquet, e può integrarsi con sistemi di archiviazione distribuita e piattaforme cloud, rendendolo uno strumento utilizzato nelle moderne architetture di elaborazione dei Big Data.



Esercizio 1 – Creazione e visualizzazione di un DataFrame

Realizzare un programma PySpark che crei un DataFrame contenente le informazioni di 5 studenti.

Per ogni studente dovranno essere presenti il nome, l'età e il voto finale.

Il programma dovrà inizializzare una sessione Spark, creare il DataFrame utilizzando dati definiti direttamente nel codice e visualizzarne il contenuto.

Successivamente, dovrà mostrare solamente le colonne relative al nome e al voto degli studenti.



Il programma inizializza una sessione Spark attraverso SparkSession e crea un DataFrame utilizzando una lista di tuple.

Il metodo createDataFrame() permette di trasformare i dati Python in una struttura tabellare distribuita.

Attraverso show() vengono visualizzati i dati, mentre select() permette di scegliere solamente le colonne desiderate.

```
from pyspark.sql import SparkSession

# Creazione della sessione Spark
spark = SparkSession.builder.appName("Studenti").getOrCreate()

# Dati degli studenti
studenti = [
    ("Mario", 20, 8),
    ("Luca", 22, 6),
    ("Anna", 19, 9),
    ("Sara", 21, 7),
    ("Paolo", 23, 5)
]

# Creazione del DataFrame
df = spark.createDataFrame(
    studenti,
    ["nome", "eta", "voto"]
)

# Visualizzazione completa
df.show()

# Selezione delle colonne
df.select("nome", "voto").show()

# Chiusura della sessione
spark.stop()
```

Esercizio 2 – Filtraggio e trasformazione dei dati

Realizzare un programma PySpark che gestisca un DataFrame contenente 6 prodotti, ciascuno caratterizzato da nome, categoria e prezzo.

Il programma dovrà utilizzare le funzionalità di PySpark per selezionare solamente i prodotti con un prezzo superiore a 50 euro.

Successivamente, dovrà creare una nuova colonna chiamata "prezzo_scontato", applicando uno sconto del 10% al prezzo originale.

Infine, dovrà visualizzare il DataFrame con i risultati ottenuti.



Il programma crea un DataFrame contenente prodotti, categorie e prezzi.

Attraverso filter() seleziona solamente i prodotti con prezzo superiore a 50 euro.

Il metodo withColumn() aggiunge una nuova colonna calcolando il prezzo scontato mediante una moltiplicazione per 0.90.

La funzione col() permette di riferirsi alle colonne del DataFrame durante le operazioni.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# Creazione della sessione Spark
spark = SparkSession.builder.appName("Prodotti").getOrCreate()

# Dati dei prodotti
prodotti = [
    ("Mouse", "Informatica", 25),
    ("Tastiera", "Informatica", 60),
    ("Monitor", "Informatica", 180),
    ("Scrivania", "Arredamento", 150),
    ("Lampada", "Arredamento", 35),
    ("Sedia", "Arredamento", 90)
]

# Creazione del DataFrame
df = spark.createDataFrame(
    prodotti,
    ["nome", "categoria", "prezzo"]
)

# Filtro dei prodotti superiori a 50 euro
df_filtrato = df.filter(col("prezzo") > 50)

# Creazione della colonna con lo sconto
df_scontato = df_filtrato.withColumn(
    "prezzo_scontato",
    col("prezzo") * 0.90
)

# Visualizzazione
df_scontato.show()

spark.stop()
```

Esercizio 3 – Analisi delle vendite con aggregazioni

Realizzare un programma PySpark che crei un DataFrame contenente almeno 8 vendite, ciascuna caratterizzata dal nome del prodotto, dalla categoria di appartenenza e dall'importo della vendita.

Il programma dovrà raggruppare i dati per categoria utilizzando groupBy() e calcolare la somma totale degli importi per ogni categoria attraverso una funzione di aggregazione.

Successivamente, dovrà ordinare i risultati in ordine decrescente rispetto al totale delle vendite e visualizzare il DataFrame finale.



Il programma crea un DataFrame contenente le vendite e utilizza groupBy() per raggruppare i prodotti in base alla categoria.

La funzione sum() calcola l'importo totale di ogni gruppo, mentre alias() assegna un nome alla colonna risultante.

Infine, orderBy() e desc() ordinano i risultati dal totale più alto a quello più basso.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import sum, col

# Creazione della sessione Spark
spark = SparkSession.builder.appName("Vendite").getOrCreate()

# Dati delle vendite
vendite = [
    ("Mouse", "Informatica", 30),
    ("Tastiera", "Informatica", 70),
    ("Monitor", "Informatica", 200),
    ("Sedia", "Arredamento", 100),
    ("Scrivania", "Arredamento", 180),
    ("Lampada", "Arredamento", 40),
    ("Penna", "Cancelleria", 5),
    ("Quaderno", "Cancelleria", 15)
]

# Creazione del DataFrame
df = spark.createDataFrame(
    vendite,
    ["prodotto", "categoria", "importo"]
)

# Raggruppamento e somma delle vendite
df_totali = df.groupBy("categoria").agg(
    sum("importo").alias("totale_vendite")
)

# Ordinamento decrescente
df_ordinato = df_totali.orderBy(
    col("totale_vendite").desc()
)

# Visualizzazione
df_ordinato.show()

spark.stop()
```


Buon davante a tutti